

Nel seguito sono riportate le istruzioni per la creazione del progetto di Laboratorio di Sistemi Operativi per il corso di Laurea in Informatica.

Gli studenti devono seguire tutte le indicazioni riportate sotto. Per tutto quanto non riportato, gli studenti possono decidere in autonomia come procedere.

Il progetto consiste nello sviluppo di un client-server. Il server deve essere sviluppato in linguaggio C, mentre siete liberi di usare un linguaggio diverso per il client (es. C, Java, Kotlin, etc..).

A prescindere dal linguaggio di implementazione del client, Client e Server devono comunicare attraverso Socket (no websocket). Client e Server possono essere locali, non è richiesto averli in remoto, ma è consentito. Il server deve gestire i client in parallelo - quindi devono essere usati thread o processi figli!

Differenze per ordinamento:

- Per il nuovo ordinamento (esame da 8 crediti): bisogna sviluppare il progetto usando Docker-compose.
- Per il vecchio ordinamento (esame da 6 crediti): non è necessario usare un docker. Sia client che server in C.

Il progetto deve essere accompagnato da:

- un README file con le istruzioni per eseguire il progetto
- una documentazione di massimo 10 pagine con la descrizione delle scelte implementative (non codice, non semplice descrizione delle funzionalità)

Il progetto verrà poi presentato e il suo funzionamento dimostrato previo appuntamento, dopo la consegna del progetto. La presentazione del progetto deve essere effettuata da tutti i componenti del progetto nello stesso giorno in presenza.

Il progetto deve essere consegnato al docente tramite email o su Teams come file zip contenente client, server, README e breve documentazione. Se per lo sviluppo del progetto è stato usato GIT, si può, in alternativa, anche dare accesso al repository che però dovrà contenere tutto il materiale richiesto.

La discussione del progetto viene stabilita dopo la consegna del progetto. Il progetto può essere consegnato in qualsiasi momento dell'anno accademico, ma le discussioni verranno pianificate in base alle consegne.

Il progetto si ritiene superato con un minimo di 18/30.

I gruppi possono essere formati da 2 componenti. Qualsiasi altro tipo di formazione per eventuali esigenze straordinarie deve essere valutata e approvata dal docente. In queste situazioni, i progetti non subiranno variazioni di complessità.

Progetto 1: Forza 4

Lo studente dovrà realizzare un server multi-client per giocare a Forza 4. Il gioco è pensato per due partecipanti. Ogni giocatore, una volta collegato, deve poter creare una o più partite, ma deve poter giocare attivamente solo a una partita alla volta. Quando un giocatore crea una nuova partita, questa viene inizialmente registrata dal server e resa disponibile agli altri utenti. Ogni partita deve essere identificata in maniera univoca, così che i client possano richiedere l'accesso indicando il relativo identificativo. Una volta creata una partita, il suo stato iniziale può essere considerato come una nuova creazione, in cui viene semplicemente registrata. Successivamente, la partita entra nello stato di attesa, durante il quale gli altri giocatori collegati al server vengono informati della sua presenza e possono chiedere di partecipare. Quando un nuovo giocatore richiede di unirsi alla partita, il creatore può accettare o rifiutare la richiesta. Solo nel caso venga accettata, la partita passa nello stato di gioco in corso e inizia effettivamente la partita di Forza 4. Durante la partita, i due giocatori vengono informati sullo stato della griglia e su chi ha il turno, mentre gli altri client collegati al server ricevono messaggi diversi e vengono solo messi a conoscenza che quella partita è in corso e non è più possibile parteciparvi. Alla conclusione della partita, lo stato diventa terminata. A questo punto il server deve inviare ai due giocatori coinvolti un esito personalizzato, distinguendo tra vittoria, sconfitta o pareggio, a seconda del risultato ottenuto. Gli altri client collegati riceveranno soltanto la notifica che la partita si è conclusa. Una volta terminata, i giocatori possono decidere se iniziare una nuova partita. In tal caso la griglia viene ripristinata e il gioco può ricominciare.

- Opzionale: il vincitore di una partita può decidere di creare una nuova sessione, diventando automaticamente il nuovo proprietario della partita se non lo era già. Il perdente, invece, è obbligato a lasciare la partita. Nel caso in cui la partita termini in pareggio, entrambi i giocatori hanno la possibilità di scegliere congiuntamente se effettuare una nuova partita. Anche per questa nuova sessione valgono le stesse regole iniziali: la partita torna nello stato di attesa e il server invita nuovamente gli altri giocatori a partecipare.

Progetto 2: Battaglia navale

Lo studente dovrà realizzare un server multi-client per giocare a Battaglia Navale. Il gioco prevede due giocatori per ogni partita. Ogni utente collegato deve essere in grado di creare una nuova partita o di unirsi a una partita già esistente. Quando un giocatore crea una partita, il server la registra assegnandole un identificativo univoco e la rende visibile a tutti gli altri client, che possono scegliere di partecipare. Come per altre tipologie di gioco, il creatore della partita può accettare o rifiutare la richiesta di partecipazione da parte di un altro giocatore. Solo dopo l'accettazione la partita può iniziare. Il gioco si divide in due fasi principali: la fase di posizionamento delle navi e la fase di combattimento. Entrambe devono essere gestite dal server. Nella fase iniziale, i due giocatori posizionano le loro navi comunicando al server le coordinate desiderate. Il server verifica l'assenza di sovrapposizioni e controlla che le navi rispettino le dimensioni e i limiti della griglia. Quando entrambi i giocatori hanno posizionato tutte le navi, la partita entra ufficialmente nella fase di gioco. Durante la partita i giocatori si alternano nel dichiarare colpi su determinate coordinate. Il server riceve la mossa, verifica se si tratta di un colpo valido e stabilisce se la cella scelta contiene una nave avversaria. La partita termina quando tutte le navi di uno dei due giocatori vengono affondate. In quel momento il server comunica l'esito finale ai due partecipanti, distinguendo tra vittoria e sconfitta. Una volta conclusa la partita, i giocatori possono scegliere se avviare una nuova sfida o se uscire definitivamente dalla sessione. Il server aggiornerà di conseguenza lo stato della partita permettendo o meno la creazione di una nuova sessione con la stessa coppia di giocatori.

- Opzionale: se un giocatore cade, l'altro giocatore può decidere di terminare la partita o aspettare che un altro giocatore si unisca

Progetto 3: Dungeon Explorer

Lo studente dovrà sviluppare un server multi-client che permetta a un gruppo da due a quattro giocatori di partecipare a un'esperienza cooperativa chiamata Dungeon Explorer. Il gioco consiste nell'esplorazione cooperativa di un dungeon generato dal server. Ogni "dungeon" rappresenta una nuova partita e deve essere identificato in modo univoco quando viene creato da uno dei giocatori. Al momento della creazione, il dungeon si trova inizialmente in uno stato passivo: il creatore diventa proprietario della partita e gli altri giocatori collegati vengono invitati a unirsi, fino a un massimo di quattro partecipanti. Il proprietario può accettare o rifiutare le richieste di ingresso, e solo quando il numero minimo di giocatori è raggiunto la partita può considerarsi pronta per iniziare. Il server ha il compito di generare la mappa del dungeon, che può essere composta da stanze collegate tra loro e caratterizzate da diversi contenuti: tesori, mostri, trappole o stanze vuote. I giocatori esplorano il dungeon insieme, comunicando al server la loro scelta di movimento o di azione. Tutte le decisioni devono essere inviate al server, che elabora lo stato del gioco e aggiorna la posizione dei giocatori, gli eventi incontrati e la progressione complessiva della partita. Gli utenti devono agire in modo sincronizzato: il server riceve le decisioni di tutti i partecipanti per ogni turno e, solo quando tutti hanno effettuato una scelta valida, procede a generare gli eventi della stanza successiva. La partita termina quando tutti gli obiettivi sono stati completati oppure quando l'intero gruppo di giocatori viene sconfitto. Una volta conclusa l'esplorazione, i partecipanti possono scegliere di intraprendere un nuovo dungeon usando la stessa squadra oppure sciogliere la partita. Se viene scelto di continuare, il server genera un nuovo dungeon e riporta la partita nello stato di attesa di nuovi giocatori.

- Opzionale: Ogni giocatore vede la lista dei tesori collezionati, mostri uccisi, e trappole trovate

Progetto 4: Sistema di guida intelligente per visitatori in un museo virtuale

Lo studente dovrà realizzare un sistema che permetta a due robot-guida di interagire con un visitatore all'interno di un museo virtuale. Il client deve essere scritto in Python oppure Kotlin, il server deve essere scritto in C. Il server e i client dovranno comunicare attraverso le sockets. I due robot fungeranno da guide digitali, ognuna con una propria personalità e un proprio stile comunicativo, e dovranno collaborare, alternarsi o completarsi durante l'interazione con l'utente. Quando un visitatore entra nel museo virtuale ed inizia a dialogare con i due robot, il server avrà il compito di analizzare le preferenze e le risposte dell'utente, così da determinarne il profilo culturale e il tipo di interesse. Le domande iniziali potrebbero riguardare le preferenze artistiche del visitatore, come la predilezione per l'arte moderna o antica, il grado di conoscenza dell'argomento, oppure lo stile di narrazione preferito, se più narrativo o più tecnico. I due robot interagiranno alternandosi nelle domande, in modo da proporre due approcci distinti: uno potrebbe essere più energico e coinvolgente, mentre l'altro potrebbe avere un tono più calmo, riflessivo e descrittivo. Il server dovrà interpretare le risposte dell'utente e, sulla base di esse, attribuirgli un profilo, come ad esempio quello di un visitatore curioso, di uno studioso interessato ai dettagli tecnici, di un

appassionato di arte contemporanea o di un visitatore occasionale che preferisce spiegazioni brevi. Una volta definito questo profilo, il server deciderà come i due robot dovranno comportarsi e come dovranno suddividersi la guida. Ad esempio, il robot più vivace potrà introdurre le opere principali, mentre quello più riflessivo potrà occuparsi degli approfondimenti storici. Oppure, nel caso di un visitatore che predilige un tono più tranquillo, sarà il robot più pacato a prendere il ruolo principale mentre l'altro interverrà solo per aggiungere note curiose. I client parlano a turno, quale tipo di contenuto deve fornire e come deve modulare la propria comunicazione. Il visitatore potrà fare domande liberamente, e queste verranno inviate al server che deciderà quale robot è più adatto a rispondere. In questo modo, i due robot potranno alternarsi o collaborare, come se formassero una coppia di guide complementari. Il server può anche decidere di far commentare lo stesso argomento a entrambi, offrendo due prospettive differenti su un'unica opera.

Il robot che si può usare è Furhat, la cui SDK è disponibile https://docs.furhat.io/getting_started/. Questo robot ha anche un emulatore.

- Opzionale: I due robot, ad esempio, potrebbero adottare stili opposti rispetto alle preferenze espresse dall'utente, come esperimento narrativo: un visitatore che preferisce spiegazioni brevi potrebbe essere guidato da robot che si dilungano in descrizioni più estese, oppure un visitatore che preferisce un tono serio potrebbe essere accolto da un robot più scherzoso, mentre l'altro mantiene un atteggiamento più formale.
- Opzionale: è possibile integrare sistemi, quali openAI, Gemini o altri (ci sono versioni gratuite disponibili per gli studenti), nel client per la generazione delle risposte, fornendo una conversazione più naturale basata sulle indicazioni comportamentali del server.